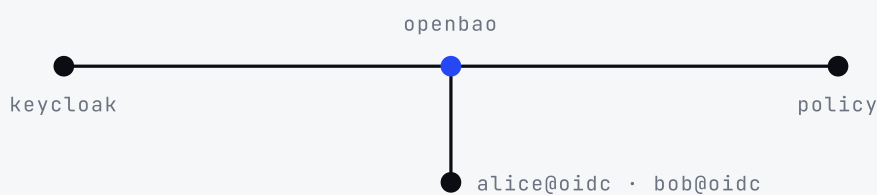


KEYCLOAK UND OPENBAO

Keycloak im Cluster, OIDC-Login für OpenBao, Gruppen
→ Policies – der Admin-Zugang ohne Passwort in
OpenBao.



Keycloak und OpenBao

Keycloak im Cluster per Operator, ein Realm als Code, OIDC-Login für OpenBao – und Gruppen, die zu Policies werden, statt Passwörtern, die in OpenBao liegen

Thomas Zachmann

17. September 2026

Inhalt

Worum es geht

- Warum von Hand, und warum ohne KI
- Lizenz und Haftung
- Ein Wort zu den Werten in diesem Leitfaden
- Für wen
- Die Bausteine
- Die Architektur in einem Bild

Teil I – Keycloak im Cluster

- Operator
- Datenbank
- TLS aus der eigenen CA
- Die Instanz

Teil II – Das Realm als Code

- Der Beweis im Token

Teil III – OIDC in OpenBao

- Auth-Methode und Konfiguration
- Zwei Rollen
- Policies
- Gruppe → External Group → Policy

Teil IV – Der Beweis

- Login mit den ID-Tokens
- Was die Rechte bewirken
- Was OpenBao angelegt hat
- Der Browser-Weg

Teil V – Was schiefging, und warum

Teil VI – Betrieb

- Den temporären Admin ablösen
- Realm-Änderungen
- Testrolle entfernen
- Nº 1, Teil VI, neu gelesen
- Was noch offen ist

Anhang A – Alle Kommandos

Anhang B – Glossar

Worum es geht

Bis hierher hatte der OpenBao im Cluster zwei Arten von Zugang: den Root-Token und die Kubernetes-Auth für Workloads. Für Menschen gab es nichts – und N° 1 empfahl als Zwischenschritt `userpass`, ein Passwort, das in OpenBao selbst liegt. Das ist besser als `root`, aber es ist ein zweites Passwort neben dem, das man ohnehin hat, ohne MFA, ohne zentrale Sperre, ohne Gruppen.

Dieser Leitfaden baut den Zugang, den man in einer Firma erwartet: Ein Identity Provider – **Keycloak** – verwaltet Benutzer und Gruppen, OpenBao vertraut ihm per OIDC, und die Gruppenzugehörigkeit in Keycloak entscheidet, welche Policies ein Token bekommt. Wer aus der Gruppe `openbao-admins` fliegt, ist beim nächsten Login kein Admin mehr – ohne dass jemand OpenBao anfasst.

Keycloak selbst wird per Operator im Cluster deployt, mit einer PostgreSQL-Datenbank aus CloudNativePG und einem TLS-Zertifikat aus der CA von N° 5. Das Realm kommt als `KeycloakRealmImport`, also als Code. Der Login wird ohne Browser bewiesen – mit einem ID-Token aus dem Password-Grant – und der Browser-Weg für UI und CLI dokumentiert.

Es ist der achte Teil einer Reihe. Vier Fehler traten auf; zwei davon lagen in Keycloak 26.7, einer in der eigenen PKI-Rolle aus N° 5.

Warum von Hand, und warum ohne KI

OIDC hat viele Stellen, an denen zwei Werte übereinstimmen müssen: Issuer und Discovery-URL, `aud` und `bound_audiences`, Redirect-URI im Client und in der Rolle, Claim-Name im Mapper und in `groups_claim`. Eine KI setzt sie konsistent – und wenn einer nicht passt, hilft sie nicht beim Finden, weil die Fehlermeldung von OpenBao nur `error validating token` sagt.

Werkzeuge wie [nyrvex](#), das die Konfiguration von Secret Store und Identity Provider einer AI-Plattform generiert, nehmen einem diese Schritte später ab. Man sollte sie einmal selbst gegangen sein, um beurteilen zu können, was generiert wurde.

Der Maßstab: Wer diesen Leitfaden durchgearbeitet hat, kann auf einem leeren Blatt zeichnen, welchen Weg eine Gruppenzugehörigkeit von Keycloak bis in `identity_policies` eines OpenBao-Tokens nimmt – über welche vier Objekte, und welches davon in welchem System lebt.

Lizenz und Haftung

Dieser Leitfaden steht unter CC BY 4.0: Er darf kopiert, weitergegeben und bearbeitet werden, auch kommerziell, solange der Autor genannt wird. Er wird ohne Gewähr bereitgestellt. Alles darin wurde in einer Entwicklungsumgebung durchgeführt; wer es in einer anderen Umgebung nachvollzieht, tut das auf eigene Verantwortung.

Ein Wort zu den Werten in diesem Leitfaden

Client-Secret (openbao-client-secret-123) und Benutzerpasswörter (alice123, bob123) sind Entwicklungswerte und bewusst abgedruckt. Das Realm-Manifest hat `directAccessGrantsEnabled: true`, damit der Test ohne Browser geht – in einer produktiven Konfiguration ist das aus.

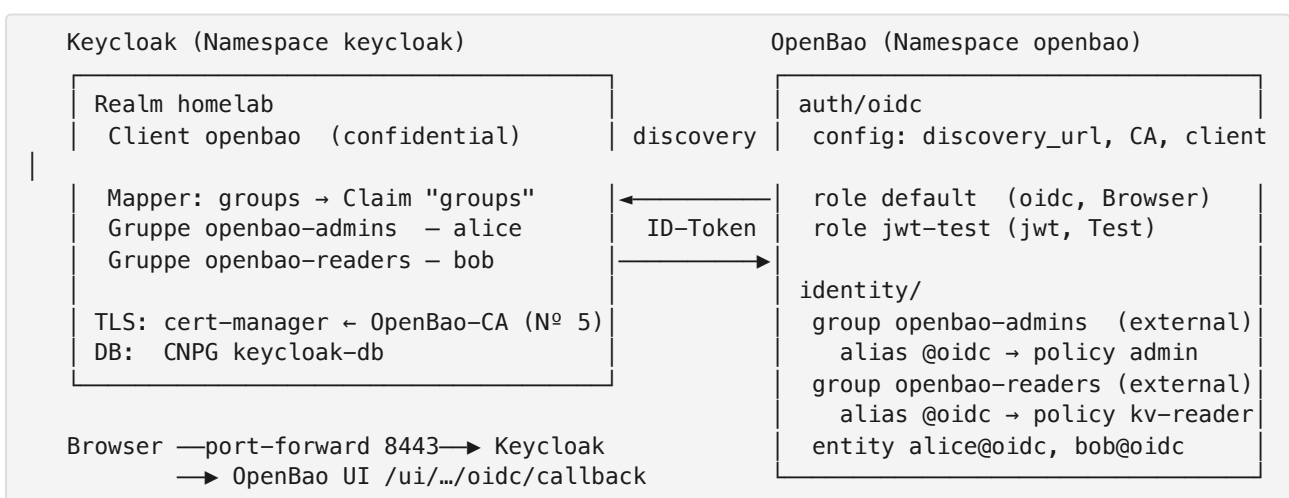
Für wen

Für Leserinnen und Leser, die N° 1 kennen und OIDC in Grundzügen verstehen (Issuer, Client, ID-Token, Claims). Keycloak-Erfahrung hilft, ist aber nicht nötig – alles Nötige steht als Manifest da. OpenBao-Grundlagen liefert mein Buch **Vault in Practice** (Leanpub, leanpub.com/vault-in-practice); für Keycloak selbst gibt es *Keycloak in Practice* aus derselben Reihe.

Die Bausteine

Baustein	Version	Rolle
Keycloak Operator + Keycloak	26.7.4	der Identity Provider, Realm als CR
CloudNativePG	Operator 0.29	PostgreSQL für Keycloak, ein Cluster-Objekt
cert-manager + OpenBao-CA (N° 5)	v1.20.2	TLS-Zertifikat für Keycloak
OpenBao im Cluster (N° 1)	2.6.2	Auth-Methode oidc, Identity-Groups

Die Architektur in einem Bild



Drei Dinge, die man aus dem Bild mitnehmen sollte:

1. **Vier Objekte tragen die Gruppe.** Die Gruppe in Keycloak, der Mapper, der sie in den Token schreibt, die External Group in OpenBao, und der Alias, der beide verbindet – gebunden an den Accessor der Auth-Methode. Fehlt eines, kommt der Login durch, aber ohne Policies.
2. **Der Hostname ist der Service-Name.** Keycloak terminiert TLS selbst mit einem Zertifikat der eigenen CA; OpenBao im Cluster und ein Browser per Port-Forward sehen denselben Issuer. Kein DNS, kein Ingress nötig.
3. **Token-Policies kommen von der Rolle, Identity-Policies von der Gruppe.** Beide zusammen sind die effektiven Rechte. Die Rolle gibt jedem default; alles Weitere entscheidet Keycloak.

Teil I – Keycloak im Cluster

Operator

```
V=26.7.4
for f in keycloaks keycloakrealmimports keycloakoidclients keycloaksamlclients; do
    kubectl apply -f https://raw.githubusercontent.com/keycloak/keycloak-k8s-resources/$V/kubernetes/$f.k8s.keycloak.org-v1.yml
done
kubectl apply -n keycloak -f https://raw.githubusercontent.com/keycloak/keycloak-k8s-resources/$V/kubernetes/kubernetes.yml
```

Fehler 1 – zwei CRDs zu wenig. Die Dokumentation nennt zwei CRD-Dateien (keycloaks, keycloakrealmimports). Mit nur diesen beiden stürzt der Operator 26.7 beim Start ab:

```
Registered reconciler: 'keycloakoidclientcontroller' for resource:
  'class org.keycloak.operator.crd.v2alpha1.client.KeycloakOIDClient'
...
Couldn't start informer for keycloakoidclients.k8s.keycloak.org/v2alpha1
```

26.7 hat zwei neue CRDs – KeycloakOIDClient und KeycloakSAMLClient – und der Operator registriert Controller dafür, ob man sie nutzt oder nicht. Alle vier CRD-Dateien der Release-Version anlegen; kustomization.yml im selben Verzeichnis listet sie.

Fehler 2 – die Startup-Probe. Danach startete der Operator weiter neu:

```
Startup probe failed: Get "http://10.42.5.182:8080/q/health/started": connection refused
Container keycloak-operator failed startup probe, will be restarted
...
keycloak-operator 26.7.4 on JVM (powered by Quarkus 3.33.3.2) started in 32.095s
```

Die Probe erlaubt $5\text{ s} + 3 \times 10\text{ s}$; die JVM brauchte auf dieser Hardware 32 Sekunden. Knapp daneben, jedes Mal. failureThreshold auf 30 hochsetzen – das ist eine Patch-Zeile am Deployment, kein Umbau.

Datenbank

CloudNativePG ist im Cluster; ein Cluster-Objekt reicht:

```

apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: keycloak-db
  namespace: keycloak
spec:
  instances: 1
  storage:
    size: 5Gi
    storageClass: longhorn
  bootstrap:
    initdb:
      database: keycloak
      owner: keycloak

```

CNPG legt das Secret `keycloak-db-app` mit `username/password` an – genau die Form, die das Keycloak-CR erwartet. Kein Passwort in einem Manifest.

TLS aus der eigenen CA

```

apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: keycloak-tls
  namespace: keycloak
spec:
  secretName: keycloak-tls
  privateKey:
    algorithm: ECDSA
    size: 256
    rotationPolicy: Always
  dnsNames:
    - keycloak-service.keycloak.svc.cluster.local
    - keycloak-service.keycloak.svc
  issuerRef:
    kind: ClusterIssuer
    name: openbao-cluster-internal

```

Fehler 3 – die eigene Rolle.

```
* subject alternate name keycloak-service.keycloak.svc not allowed by this role
```

Die PKI-Rolle aus N° 5 erlaubt `svc.cluster.local` mit Subdomains und `demo.svc` – aber nicht `keycloak.svc`. Kurznamen sind pro Namespace; jeder neue Namespace, der Kurznamen im Zertifikat will, muss in `allowed_domains` nachgetragten werden. Das ist kein Bug, das ist die Rolle, die tut, wofür sie da ist. Nachtrag:

```

/ $ bao write pki_int/roles/cluster-internal \
    allowed_domains="svc.cluster.local,demo.svc,keycloak.svc,example.internal" ...

```

`cert-manager` wartet nach einem Fehler mit Backoff. Wer nicht warten will, löscht das Certificate und legt es neu an.

Die Instanz

```
apiVersion: k8s.keycloak.org/v2beta1
kind: Keycloak
metadata:
  name: keycloak
  namespace: keycloak
spec:
  instances: 1
  db:
    vendor: postgres
    host: keycloak-db-rw
    database: keycloak
    usernameSecret: { name: keycloak-db-app, key: username }
    passwordSecret: { name: keycloak-db-app, key: password }
  hostname:
    hostname: https://keycloak-service.keycloak.svc.cluster.local:8443
    strict: true
  http:
    tlsSecret: keycloak-tls
  ingress:
    enabled: false
```

Der Hostname ist der eigene Service-Name mit Port. Das ist die Entscheidung, die den Rest einfach macht: OpenBao im Cluster erreicht `keycloak-service.keycloak.svc.cluster.local:8443` direkt, und ein Browser außerhalb erreicht denselben Namen per `kubectl port-forward` und einem Eintrag in `/etc/hosts`. Kein Ingress, kein DNS, und der Issuer im Token ist für beide identisch – das ist die Bedingung, an der OIDC sonst scheitert.

```
$ kubectl get keycloak -n keycloak -o jsonpath='{.status.conditions[?(@.type=="Ready")].status}'
True
```

Discovery, aus einem Pod im Cluster, mit dem Root der OpenBao-CA:

```
$ curl --cacert /ca/ca.crt https://keycloak-service.keycloak.svc.cluster.local:8443/realms/master/.well-known/openid-configuration
{"issuer":"https://keycloak-service.keycloak.svc.cluster.local:8443/realms/master", ...}
```

TLS verifiziert gegen die CA aus N° 5. Das ist der Moment, in dem sich die eigene PKI auszahlt.

Teil II – Das Realm als Code

```
apiVersion: k8s.keycloak.org/v2beta1
kind: KeycloakRealmImport
metadata:
  name: homelab
  namespace: keycloak
spec:
  keycloakCRName: keycloak
  realm:
    realm: homelab
    enabled: true
    groups:
      - name: openbao-admins
      - name: openbao-readers
    clients:
      - clientId: openbao
        publicClient: false
        secret: openbao-client-secret-123
        standardFlowEnabled: true
        directAccessGrantsEnabled: true      # nur für den Test ohne Browser
        redirectUris:
          - "https://bao.example.internal/ui/vault/auth/oidc/oidc/callback"
          - "http://localhost:8250/oidc/callback"
        protocolMappers:
          - name: groups
            protocol: openid-connect
            protocolMapper: oidc-group-membership-mapper
            config:
              claim.name: groups
              full.path: "false"
              id.token.claim: "true"
              access.token.claim: "true"
    users:
      - username: alice
        enabled: true
        email: alice@example.internal
        firstName: Alice
        lastName: Anders
        emailVerified: true
        requiredActions: []
        groups: ["openbao-admins"]
        credentials:
          - type: password
            value: alice123
            temporary: false
      - username: bob
        ...
        groups: ["openbao-readers"]
```

Drei Entscheidungen darin:

- **Der Mapper ist der Kern.** Ohne `oidc-group-membership-mapper` steht keine Gruppe im Token, und OpenBao hat nichts, worauf es Policies abbilden könnte. `full.path: "false"` liefert `openbao-admins` statt `/openbao-admins` – der Alias in OpenBao muss exakt so heißen.

- **Zwei Redirect-URLs.** Die erste für die OpenBao-UI (der Pfad ist fest vorgegeben: /ui/vault/auth/oidc/oidc/callback – ja, vault, die UI hat den Pfad behalten), die zweite für bao login –method=oidc, das einen lokalen Listener auf Port 8250 öffnet.
- **emailVerified: true und requiredActions: [].** Das ist Fehler 4:

```
{"error":"invalid_grant","error_description":"Account is not fully set up"}
```

Ein importierter Benutzer ohne diese Felder hat offene Required Actions (E-Mail verifizieren, Profil vervollständigen) und kann sich nicht anmelden, bis er sie im Browser erledigt hat. Für einen Test-User per Password-Grant gibt es diesen Browser nicht.

Und ein Detail, das man wissen muss: Ein KeycloakRealmImport **überschreibt nicht**. Existiert das Realm, tut der Import nichts. Nach einer Korrektur am Manifest: Realm per Admin-API löschen, CR löschen, CR neu anlegen.

```
$ kubectl get keycloakrealmimport -n keycloak homelab -o jsonpath='{.status.conditions[?(@.type=="Done")].status}'
True
```

Der Beweis im Token

Ein ID-Token für alice per Password-Grant, aus einem Pod im Cluster:

```
$ curl --cacert /ca/ca.crt \
  -d client_id=openbao -d client_secret=openbao-client-secret-123 \
  -d grant_type=password -d username=alice -d password=alice123 -d scope=openid \
  https://keycloak-service.keycloak.svc.cluster.local:8443/realms/homelab/protocol/openid-connect/token
```

Der Payload des id_token, dekodiert:

```
iss:                https://keycloak-service.keycloak.svc.cluster.local:8443/realms/homelab
aud:                openbao
preferred_username: alice
groups:             ["openbao-admins"]
```

Vier Werte, die gleich in OpenBao gebraucht werden: iss muss zur oidc_discovery_url passen, aud zu bound_audiences, preferred_username wird user_claim, groups wird groups_claim.

Teil III – OIDC in OpenBao

Auth-Methode und Konfiguration

```
/ $ bao auth enable oidc
/ $ bao write auth/oidc/config \
  oidc_discovery_url="https://keycloak-service.keycloak.svc.cluster.local:8443/realms/
homelab" \
  oidc_discovery_ca_pem=@/tmp/root.pem \
  oidc_client_id=openbao \
  oidc_client_secret=openbao-client-secret-123 \
  default_role=default
```

oidc_discovery_ca_pem ist die Stelle, an der N° 5 und N° 8 sich berühren: OpenBao vertraut Keycloaks Zertifikat, weil es die eigene Root-CA kennt. Ohne diese Zeile: x509: certificate signed by unknown authority.

Zwei Rollen

```
/ $ bao write auth/oidc/role/default \
  role_type=oidc user_claim=preferred_username groups_claim=groups \
  bound_audiences=openbao oidc_scopes=openid \
  allowed_redirect_uris="https://bao.example.internal/ui/vault/auth/oidc/oidc/callback
k,http://localhost:8250/oidc/callback" \
  token_policies=default token_ttl=1h token_max_ttl=8h

/ $ bao write auth/oidc/role/jwt-test \
  role_type=jwt user_claim=preferred_username groups_claim=groups \
  bound_audiences=openbao token_policies=default token_ttl=15m
```

Die Auth-Methode oidc kann zwei Rollentypen: oidc für den Browser-Flow (Redirect, Code, Callback) und jwt für ein Token, das man schon hat. Die zweite Rolle ist nur für den Test hier – sie erlaubt jedem mit einem gültigen ID-Token dieses Realms den Login, ohne Browser. Nach dem Test löschen.

token_policies=default ist Absicht: Die Rolle gibt **keine** Rechte. Die kommen aus den Gruppen.

Policies

```
/ $ bao policy write kv-reader - <<'EOF'
path "secret/data/demo/*"      { capabilities = ["read"] }
path "secret/metadata/demo/*"  { capabilities = ["read", "list"] }
path "secret/metadata/demo"    { capabilities = ["list"] }
EOF
```

admin ist die Policy aus N° 1, Teil VI – hier erstmals im Cluster angelegt. Sie ersetzt damit nicht den Root-Token; das ist die Härtung, die in N° 1 beschrieben ist und auf diesem Cluster noch aussteht.

Gruppe → External Group → Policy

```
/ $ ACC=$(bao read -field=accessor sys/auth/oidc)
/ $ ID=$(bao write -field=id identity/group name=openbao-admins type=external policies=admin)
/ $ bao write identity/group-alias name=openbao-admins mount_accessor=$ACC canonical_id=$ID
```

Dasselbe für openbao-readers → kv-reader. Das Ergebnis:

```
/ $ bao read identity/group/name/openbao-admins
openbao-admins -> ['admin'] | alias: openbao-admins
openbao-readers -> ['kv-reader'] | alias: openbao-readers
```

Eine **External Group** hat keine Mitglieder in OpenBao. Ihre Mitglieder kommen von außen: Beim Login liest OpenBao `groups_claim` aus dem Token, sucht für jeden Wert einen Group-Alias mit diesem Namen am Accessor der Auth-Methode, und hängt die Entity des Benutzers für die Dauer des Tokens in diese Gruppe. Die Policies der Gruppe werden `identity_policies`.

Der Alias-Name muss dem Claim-Wert **exakt** entsprechen – `openbao-admins`, nicht `/openbao-admins` (daher `full.path: "false"` im Mapper).

Ein Stolperstein aus dem Terminal, kein OpenBao-Thema: In zsh ist `GID` eine reservierte Variable (die Gruppen-ID des Prozesses). `GID=$(bao write -field=id ...)` scheitert mit `bad math expression`. Anderer Name.

Teil IV – Der Beweis

Login mit den ID-Tokens

```
== alice
/ $ bao write -format=json auth/oidc/login role=jwt-test jwt=<id_token>
  token_policies      ['default']
  identity_policies   ['admin']
  effective            ['admin', 'default']

== bob
  token_policies      ['default']
  identity_policies   ['kv-reader']
  effective            ['default', 'kv-reader']
```

Genau die Trennung, die das Bild oben zeigt: token_policies von der Rolle (für beide default), identity_policies von der Gruppe. Kein Passwort, keine Rolle, kein Benutzer in OpenBao wurde für alice oder bob angelegt.

Was die Rechte bewirken

```
== bob liest ein Secret:
/ $ BAO_TOKEN=<bob> bao kv get -field=admin_username secret/demo/miniflux
admin
== bob schreibt:
/ $ BAO_TOKEN=<bob> bao kv put secret/demo/miniflux x=y
* permission denied
== bob listet Policies:
* permission denied
== alice listet Policies:
admin cert-manager default eso-demo kv-reader snapshot root
```

Was OpenBao angelegt hat

```
/ $ bao list identity/entity/name
entity_0b8b5371.root | aliases ['alice@oidc'] | groups 1
entity_3584c432.root | aliases ['bob@oidc']   | groups 1
entity_...           | aliases ['...@kubernetes'] | groups 0      (ESO, cert-manager, Snapsho
t-Job)
```

Beim ersten Login entsteht pro Benutzer eine **Entity** mit einem Alias <user_claim>@oidc. Sie bleibt bestehen; ändert sich die Gruppe in Keycloak, ändert sich beim nächsten Login die Gruppenzugehörigkeit der Entity – und damit die Policies. Die drei Entities ohne Gruppen sind die Workloads aus N° 2, 5 und 7.

Der Browser-Weg

Für UI und CLI muss der Browser Keycloak erreichen:

```
# /etc/hosts auf dem Arbeitsplatz
127.0.0.1 keycloak-service.keycloak.svc.cluster.local

$ kubectl port-forward -n keycloak svc/keycloak-service 8443:8443
```

und der Root-CA aus N° 5 vertrauen (Browser oder Betriebssystem). Dann:

```
$ export BAO_ADDR=https://bao.example.internal
$ bao login -method=oidc role=default
Complete the login via your OIDC provider. Launching browser to:
  https://keycloak-service.keycloak.svc.cluster.local:8443/realms/home/protocol/openid-connect/auth?...
Waiting for OIDC authentication to complete...
```

Der Browser landet bei Keycloak, alice meldet sich an, Keycloak leitet auf `http://localhost:8250/oidc/callback` zurück, die CLI hat den Token. In der UI: Methode **OIDC**, Rolle default, Button „Sign in with OIDC Provider“.

Dieser Teil ist hier nicht ausgeführt – er braucht einen Browser, sudo für `/etc/hosts` und den Vertrauensanker im Betriebssystem. Die Rolle default und die Redirect-URLs sind aber genau dafür konfiguriert; der JWT-Test in Teil IV prüft alles außer dem Redirect selbst.

Teil V – Was schiefging, und warum

Fehler	Meldung	Ursache	Lösung
1	Operator-Crash: Couldn't start informer for keycloakoidclients...	26.7 hat vier CRDs, die Doku nennt zwei	alle CRD-Dateien der Release anlegen
2	failed startup probe, will be restarted	JVM braucht 32 s, Probe erlaubt 35 s inkl. Connect- Refused-Phase	startupProbe.failureThreshold erhöhen
3	subject alternate name keycloak- service.keycloak.svc not allowed by this role	PKI-Rolle (N° 5) kennt demo.svc, nicht keycloak.svc	Namespace in allowed_domains ergänzen
4	Account is not fully set up	importierte User mit offenen Required Actions	emailVerified: true, requiredActions: []
–	bad math expression bei GID=...	GID ist in zsh reserviert	anderer Variablenname

Und das Muster: Drei der vier Fehler kamen **vor** dem ersten OIDC-Login – Operator, Zertifikat, Benutzer. OIDC selbst hat beim ersten Versuch funktioniert, weil die vier Paare (Issuer/Discovery, aud/bound_audiences, Redirect-URIs, Claim-Name) vorher im Token geprüft wurden. Wer den ID-Token dekodiert, bevor er OpenBao konfiguriert, erspart sich das Raten bei `error validating token`.

Teil VI – Betrieb

Den temporären Admin ablösen

Keycloak 26 legt beim ersten Start einen **temporären** Admin an (keycloak-initial-admin, Benutzer temp-admin). Er ist für genau eine Aufgabe da: einen dauerhaften Admin anlegen. Danach den temporären löschen. Wer das vergisst, hat einen Admin mit einem Passwort in einem Kubernetes-Secret – dasselbe Problem, das dieser Leitfaden für OpenBao löst.

Realm-Änderungen

KeycloakRealmImport ist ein Import, keine Reconciliation. Für laufende Änderungen – neue Gruppe, neuer Benutzer – gibt es drei Wege: die Admin-Konsole (nicht als Code), die Admin-REST-API in einem Skript, oder die neuen CRs KeycloakOIDCClient (26.7) für Clients. Gruppen und Benutzer haben noch kein CR; für ein Homelab ist die Konsole ehrlich genug, für eine Firma ist es Terraform mit dem Keycloak-Provider.

Testrolle entfernen

```
/ $ bao delete auth/oidc/role/jwt-test
```

und directAccessGrantsEnabled: false im Client. Beides zusammen war der Weg, ohne Browser zu testen; in Betrieb ist es eine Umgehung des Browser-Flows – und damit von MFA, falls Keycloak es verlangt.

Nº 1, Teil VI, neu gelesen

Mit OIDC ist userpass nicht mehr der Ersatz für den Root-Token, sondern nur noch **Break Glass**: der Weg hinein, wenn Keycloak nicht erreichbar ist. Die admin-Policy existiert jetzt; was fehlt, ist derselbe Schritt wie zuvor – den Ersatzzugang beweisen, dann root widerrufen. Nur dass der Ersatzzugang jetzt bao login -method=oidc heißt, und der Break-Glass-Weg userpass mit einem Passwort im Passwort-Manager.

Was noch offen ist

- **Kein MFA.** Keycloak kann OTP verlangen; für openbao-admins gehört es an. Das ist Keycloak-Konfiguration, nicht OpenBao.
- **Kein Audit-Device** (Nº 1). Wer sich wann per OIDC angemeldet hat, steht in Keycloaks Events – und nicht in OpenBao.
- **Der Browser-Weg** ist konfiguriert, nicht ausgeführt.

- **Client-Secret im Manifest.** Für die Produktion: Keycloak erzeugt es, OpenBao bekommt es per ExternalSecret... aus OpenBao. Das ist ein Zirkel, der sich nur mit einem manuellen Bootstrap-Schritt auflöst.

Anhang A – Alle Kommandos

```
# — Keycloak —————
V=26.7.4
for f in keycloaks keycloakrealmimports keycloakoidcclients keycloaksamlclients; do
    kubectl apply -f https://raw.githubusercontent.com/keycloak/keycloak-k8s-resources/$V/kubernetes/$f.k8s.keycloak.org-v1.yml
done
kubectl create ns keycloak
kubectl apply -n keycloak -f https://raw.githubusercontent.com/keycloak/keycloak-k8s-resources/$V/kubernetes/kubernetes.yml
kubectl patch deploy -n keycloak keycloak-operator --type=json \
    -p='[{"op":"replace","path":"/spec/template/spec/containers/0/startupProbe/failureThreshold","value":30}]'
kubectl apply -f k8s/keycloak-db.yaml
bao write pki_int/roles/cluster-internal allowed_domains="svc.cluster.local,demo.svc,keycloak.svc,example.internal" ...
kubectl apply -f k8s/keycloak.yaml
kubectl apply -f k8s/realm-homelab.yaml
kubectl get keycloak,keycloakrealmimport -n keycloak

# — OpenBao (siehe openbao/oidc-setup.sh) —————
bao read -field=certificate pki/cert/ca > /tmp/root.pem
bao auth enable oidc
bao write auth/oidc/config oidc_discovery_url=... oidc_discovery_ca_pem=@/tmp/root.pem oidc_client_id=openbao oidc_client_secret=... default_role=default
bao write auth/oidc/role/default role_type=oidc user_claim=preferred_username groups_claim=groups bound_audiences=openbao allowed_redirect_uris=... token_policies=default
bao policy write kv-reader - < kv-reader.hcl
ACC=$(bao read -field=accessor sys/auth/oidc)
ID=$(bao write -field=id identity/group name=openbao-admins type=external policies=admin)
bao write identity/group-alias name=openbao-admins mount_accessor=$ACC canonical_id=$ID

# — Test ohne Browser —————
curl --cacert root.pem -d client_id=openbao -d client_secret=... -d grant_type=password \
    -d username=alice -d password=alice123 -d scope=openid https://keycloak-service.keycloak.svc.cluster.local:8443/realms/homelab/protocol/openid-connect/token
bao write auth/oidc/login role=jwt-test jwt=<id_token>

# — Browser (Arbeitsplatz) —————
kubectl port-forward -n keycloak svc/keycloak-service 8443:8443
bao login -method=oidc role=default
```

Anhang B – Glossar

OIDC – OpenID Connect: Login-Protokoll auf OAuth 2.0, das einen signierten ID-Token mit Claims über den Benutzer liefert.

Issuer / Discovery – Der Issuer ist die URL, die im Token als iss steht; unter <issuer>/well-known/openid-configuration liegt die Konfiguration samt Schlüsseln. Beide müssen übereinstimmen.

Claim – Ein Feld im Token (preferred_username, groups, aud). Keycloak-Mapper schreiben Claims; OpenBao liest sie über user_claim und groups_claim.

Client – Die registrierte Anwendung in Keycloak (hier openbao), mit Secret und erlaubten Redirect-URIs.

Password Grant – OAuth-Flow, bei dem Benutzername und Passwort direkt gegen ein Token getauscht werden. Für Tests; umgeht MFA.

Rolle (OIDC-Auth) – OpenBao-Objekt, das festlegt, wie ein Token geprüft wird und welche token_policies es gibt. role_type oidc (Browser) oder jwt (vorhandenes Token).

Entity / Alias – OpenBaos Identität eines Benutzers und ihre Verknüpfung mit einer Auth-Methode (alice@oidc).

External Group – Identity-Gruppe ohne eigene Mitglieder; Zugehörigkeit kommt beim Login aus dem groups_claim, über einen Group-Alias am Accessor der Auth-Methode.

identity_policies – Policies, die ein Token über die Gruppen seiner Entity bekommt – zusätzlich zu den token_policies der Rolle.

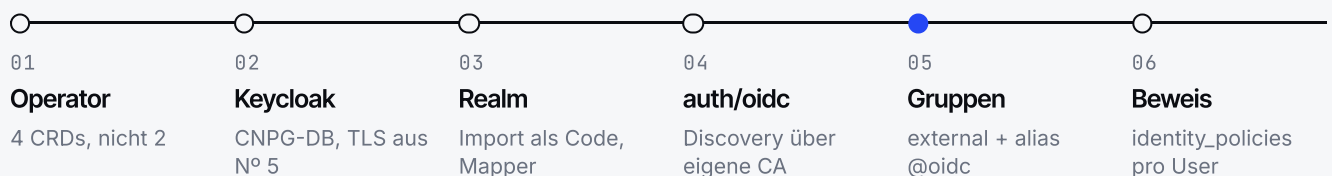
KeycloakRealmImport – CR des Keycloak-Operators, das ein Realm einmalig importiert. Kein Overwrite, keine Reconciliation.

Wer aus der Gruppe fliegt, ist beim nächsten Login kein Admin mehr. Niemand fasst OpenBao an.

Dieser Leitfaden deployt Keycloak per Operator mit einer CNPG-Datenbank und einem Zertifikat aus der eigenen CA, importiert ein Realm als Code – Client, Group-Mapper, Gruppen, Benutzer – und bindet OpenBao per OIDC daran. Die Gruppenzugehörigkeit wird über External Groups zu Policies.

Bewiesen mit ID-Tokens: alice bekommt `identity_policies [admin]`, bob `[kv-reader]`, bob liest und darf nicht schreiben. Vier Fehler unterwegs, drei davon vor dem ersten Login: zwei fehlende CRDs, eine zu knappe Startup-Probe, die eigene PKI-Rolle, Account is not fully set up.

```
-- Vier Objekte tragen die Gruppe
Keycloak: Gruppe openbao-admins → Mapper "groups"
OpenBao: identity/group openbao-admins (external, policies=admin)
         identity/group-alias openbao-admins @ accessor von auth/oidc
```



keycloak	26.7.4 · operator · realm import
database	CNPG 0.29 · 1 Instanz
tls	cert-manager · OpenBao-CA
openbao	2.6.2 · auth/oidc · identity
policies	admin · kv-reader

21 Seiten · 6 Teile · 2 Anhänge
Vertiefung: **Vault in Practice**
leanpub.com/vault-in-practice
Gesetzt in Inter Tight, Inter und JetBrains Mono.
Alle Ausgaben sind echte Terminal-Ausgaben.
CC BY 4.0 · Weitergabe erlaubt · ohne Gewähr